Neural Network-Based Market Direction Classification: A Comparative Study of Logistic Regression, Feedforward Neural Networks, and LSTM

Armaan Singh
I^2 Mathematics
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)
Kerala, India
armaan24@iisertvm.ac.in

***Abstract*—This paper investigates whether features derived from historical NIFTY 50 index prices can classify the direction of the next trading session. A majority-class baseline and Logistic Regression are compared with a feedforward neural network (FNN) and a Long Short-Term Memory (LSTM) network. The feature set contains lagged returns, rolling volatility, moving-average ratios, and momentum indicators. The data are divided chronologically into training, validation, and held-out test partitions, and feature standardization is fitted only on the training partition. On the test set, the LSTM obtains an accuracy of 50.65% and a positive-class F1-score of 0.6717, marginally above the majority baseline. However, its predictions are strongly biased toward the upward class, so the apparent advantage does not demonstrate reliable discrimination between upward and downward sessions. A simplified long-or-cash backtest produces a 1.06% cumulative return for the LSTM under an assumed one-way transaction cost of 0.1%, compared with 0.47% for buy-and-hold. These findings are exploratory and do not establish a robust trading edge.**

***Index Terms*—Financial time series, market direction prediction, feedforward neural network, LSTM, Logistic Regression, backtesting, NIFTY 50.**

I. Introduction

Forecasting financial-market direction is challenging because price movements are noisy, non-stationary, and influenced by many interacting factors. Machine-learning methods can model nonlinear relationships in historical observations, but model complexity alone does not guarantee predictive value. This study compares a conventional classifier with two neural-network architectures and evaluates their outputs using both classification metrics and a simplified trading simulation.

The research question is: *Do engineered features derived from historical NIFTY 50 prices improve next-session direction classification over simple baselines, and does any observed advantage persist in a basic trading simulation?*

II. Data and Target Definition

Historical NIFTY 50 index data were retrieved using the `yfinance` Python package. Following feature construction and removal of rows with missing values, the model dataset contained 4,092 observations. The final dates represented in the notebook extend through 29 September 2026.

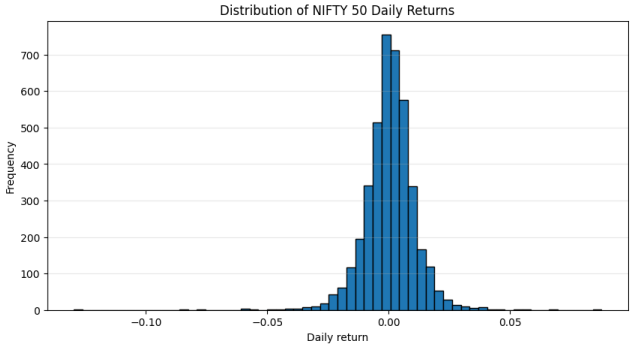


Fig. 1. Exploratory visualization of historical returns and class distribution.

The binary target is defined as

$$y_t = \begin{cases} 1, & C_{t+1} > C_t, \\ 0, & C_{t+1} \le C_t, \end{cases} \tag{1}$$

where C_t denotes the closing price at session t. The final observation, for which the next closing price is unavailable, is excluded from supervised learning. The target encodes direction only, not the magnitude of the move or its economic significance after transaction costs.

III. Feature Engineering

Twelve features were derived from historical prices and returns:

- **Lagged returns:** returns at lags 1–5 sessions.
- **Rolling volatility:** rolling standard deviations of returns over 5, 10, and 20 sessions.
- **Moving-average ratios:** current close relative to 5- and 20-session moving averages, minus one.
- **Momentum:** close-to-close percentage changes over 5 and 10 sessions.

TABLE I
CHRONOLOGICAL DATA SPLIT

Partition	Observations	Date range
Training	2,864	2010-02-02–2021-10-08
Validation	614	2021-10-11–2024-04-04
Test	614	2024-04-05–2026-09-29

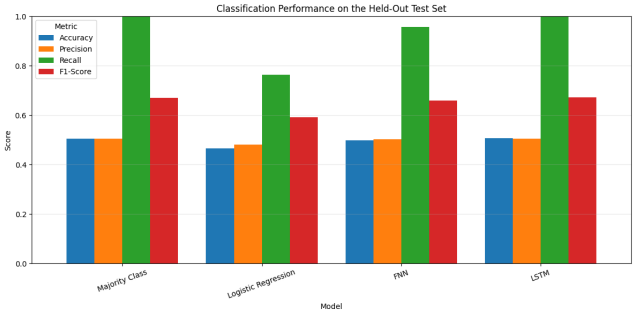


Fig. 2. Classification metric comparison on the held-out test set.

The feature set does not include macroeconomic variables, company fundamentals, news sentiment, or order-book data.

IV. EXPERIMENTAL SETUP

A. Chronological Split

The data were divided in time order into 70% training, 15% validation, and 15% test partitions. The resulting sizes and date ranges are shown in Table I.

A `StandardScaler` was fitted on training features only and then applied to validation and test features. The FNN contains hidden layers of 64 and 32 units with ReLU activations and a single output logit. The LSTM uses a 20-session input sequence and hidden dimension 64. Both neural networks were trained with Adam and binary cross-entropy with logits, using validation loss for early stopping. In the stored run, both neural networks selected epoch 1 as their best validation-loss epoch.

B. Models

Four approaches were evaluated:

1) **Majority baseline:** predicts the most frequent training class.
2) **Logistic Regression:** provides a conventional machine-learning baseline.
3) **FNN:** learns nonlinear relationships among the engineered features.
4) **LSTM:** processes sequences of 20 feature vectors to represent temporal context.

V. CLASSIFICATION RESULTS

Table II reports the held-out test metrics stored in the notebook.

The LSTM exceeds the majority baseline in accuracy by only 0.16 percentage points. Its recall for the positive class is 1.00, and the notebook's confusion-matrix output indicates that it predicts nearly all test observations as upward. The

TABLE II
HELD-OUT TEST CLASSIFICATION RESULTS

Model	Accuracy	Precision	Recall	F1-score
Majority baseline	0.5049	0.5049	1.0000	0.6710
Logistic Regression	0.4658	0.4817	0.7645	0.5910
FNN	0.4984	0.5017	0.9581	0.6585
LSTM	**0.5065**	0.5057	1.0000	**0.6717**

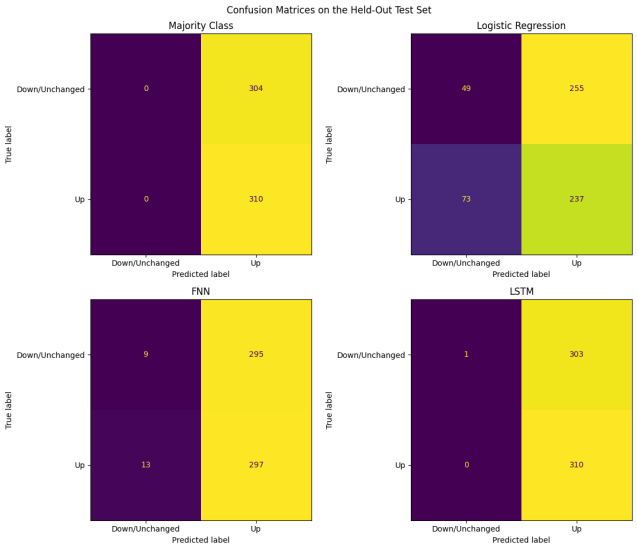


Fig. 3. Walk-forward validation results for Logistic Regression.

FNN also strongly favors upward predictions. Thus, the high positive-class recall and F1-scores should not be interpreted as balanced direction classification. Logistic Regression performs below the majority baseline on both accuracy and F1-score.

Accuracy and positive-class F1 alone are insufficient for a complete evaluation when predicted classes are highly imbalanced. Balanced accuracy, macro-F1, Matthews correlation coefficient, and predicted class proportions would provide additional insight.

VI. WALK-FORWARD VALIDATION

An expanding-window `TimeSeriesSplit` evaluation was conducted for Logistic Regression on the development period through validation data. Across five folds, mean accuracy was 0.5340 (standard deviation 0.0179) and mean F1-score was 0.6144 (standard deviation 0.0571). The fold-specific majority baseline achieved mean accuracy 0.5413 and mean F1-score 0.7022. Logistic Regression did not exceed the majority baseline in F1-score in any fold.

Walk-forward retraining was not performed for the FNN and LSTM in this notebook; therefore, these results cannot establish that the neural models are more robust over time.

VII. TRADING SIMULATION

The notebook maps predicted probabilities to a long-or-cash position using a threshold of 0.50. A one-way transaction cost

TABLE III
BACKTEST PERFORMANCE ON THE HELD-OUT PERIOD

Strategy	Total return	Ann. return	Ann. volatility	Sharpe ratio	Max. drawdown	Ann. turnover
FNN	-10.42%	-4.42%	12.99%	-0.2825	-18.85%	16.0065
LSTM	1.06%	0.43%	13.63%	0.0999	-15.19%	1.2313
Logistic Regression	-31.51%	-14.39%	11.94%	-1.2410	-37.08%	75.1075
Buy and Hold	0.47%	0.19%	13.65%	0.0825	-15.77%	0.0000

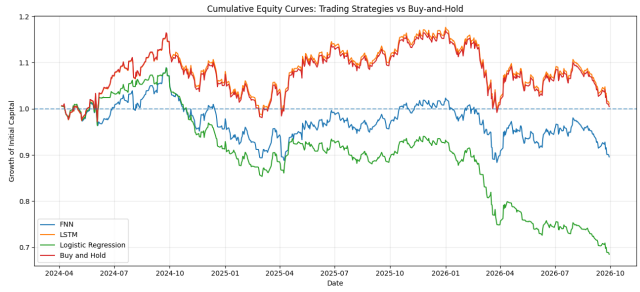


Fig. 4. Equity curves for the strategies in the held-out backtest.

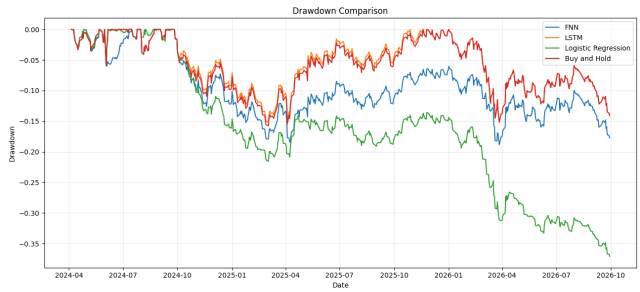


Fig. 5. Drawdown comparison across backtested strategies.

of 0.1% is deducted when position turnover occurs. Table III summarizes the stored backtest results for the held-out period.

Under the selected assumptions, the LSTM strategy returns 1.06%, compared with 0.47% for buy-and-hold. The LSTM Sharpe ratio is 0.0999, only slightly above the benchmark's 0.0825. Its maximum drawdown is 15.19%, compared with 15.77% for buy-and-hold. This small difference does not establish a reliable trading edge. The FNN and Logistic Regression strategies return -10.42% and -31.51%, respectively.

The cost-sensitivity analysis reports that the LSTM cumulative return falls from 1.36% at zero assumed cost to 0.76% at a 0.2% one-way cost. The notebook does not charge turnover costs to buy-and-hold, so cost treatment is not fully symmetric across strategies.

VIII. LIMITATIONS

Several limitations affect interpretation:

- **Weak classification signal:** test accuracy is close to 50%, and neural models predict the upward class disproportionately often.
- **Execution timing:** the backtest uses close-to-close returns following a signal based on the current close. Trading at that same close after observing it may not be feasible.
- **Simplified costs:** slippage, taxes, liquidity constraints, and interest earned on cash are not modeled.
- **Limited walk-forward coverage:** walk-forward validation is reported for Logistic Regression only.
- **Restricted feature space:** the study uses one index and price-derived features.
- **Model training:** both neural models select epoch 1 by validation loss in the saved run, which may indicate weak learning, underfitting, or limited signal.

IX. FUTURE WORK

Future experiments should add balanced accuracy, macro-F1, Matthews correlation coefficient, ROC-AUC, and class-proportion diagnostics. Thresholds should be selected on validation data only. A more realistic backtest should generate a signal after the close and execute at the next session's open, with consistent transaction costs for all strategies. Expanding-window retraining should be conducted for FNN and LSTM, using the same folds as Logistic Regression. Additional assets, periods, simple trading rules, and uncertainty estimates would help establish whether any observed advantage is robust.

X. CONCLUSION

This project implements an end-to-end pipeline for NIFTY 50 next-session direction classification, including feature engineering, chronological evaluation, baseline models, FNN and LSTM training, walk-forward validation, and a simplified backtest. The LSTM has the highest test accuracy (50.65%) and positive-class F1-score (0.6717), but its near-always-up predictions make the advantage over the majority baseline negligible. Logistic Regression does not consistently outperform the majority baseline in walk-forward validation.

The LSTM backtest is modestly positive under the chosen assumptions, but the small total return and Sharpe ratio do not demonstrate dependable trading value. The current features and model configurations do not show strong or robust market-direction predictability. The next priorities are balanced diagnostic metrics, realistic execution assumptions, and fair walk-forward comparisons across all models.

REPRODUCIBILITY NOTE

The numerical results in this paper are transcribed from the saved outputs in the submitted notebook. Before submission, verify the plots and tables against the final notebook and add references required by the course or institution.

REFERENCES

[1] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[4] Ran Aroussi, "yfinance: Download market data from Yahoo! Finance's API," software documentation. [Online]. Available: https://github.com/ranaroussi/yfinance